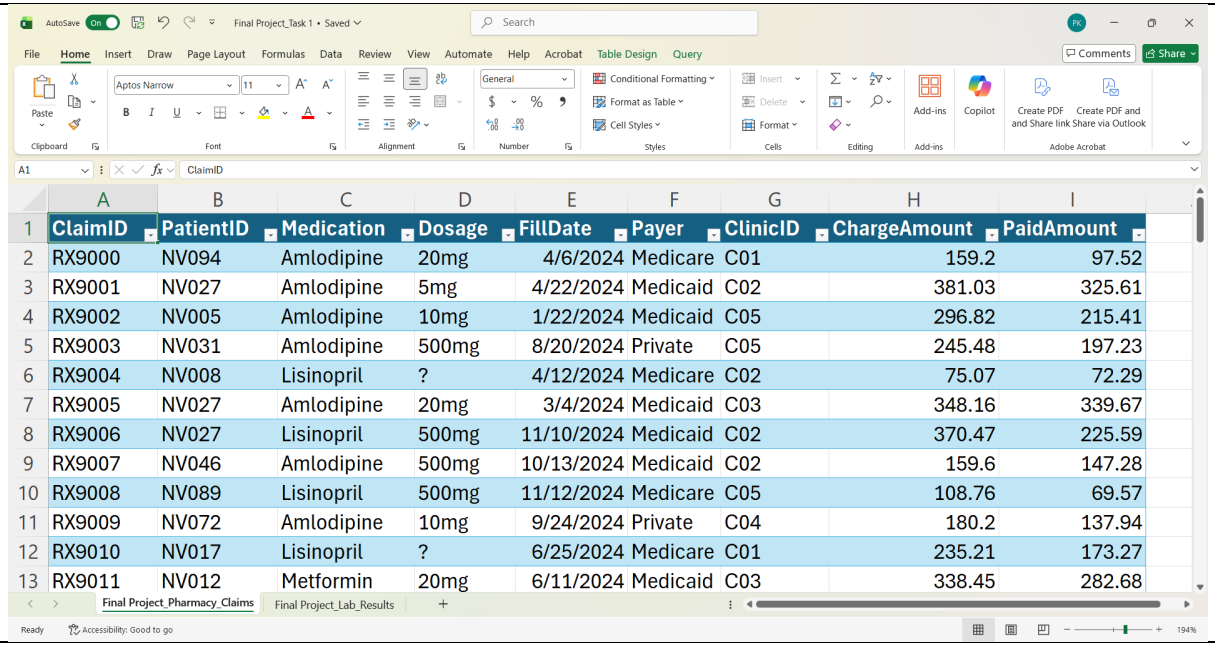
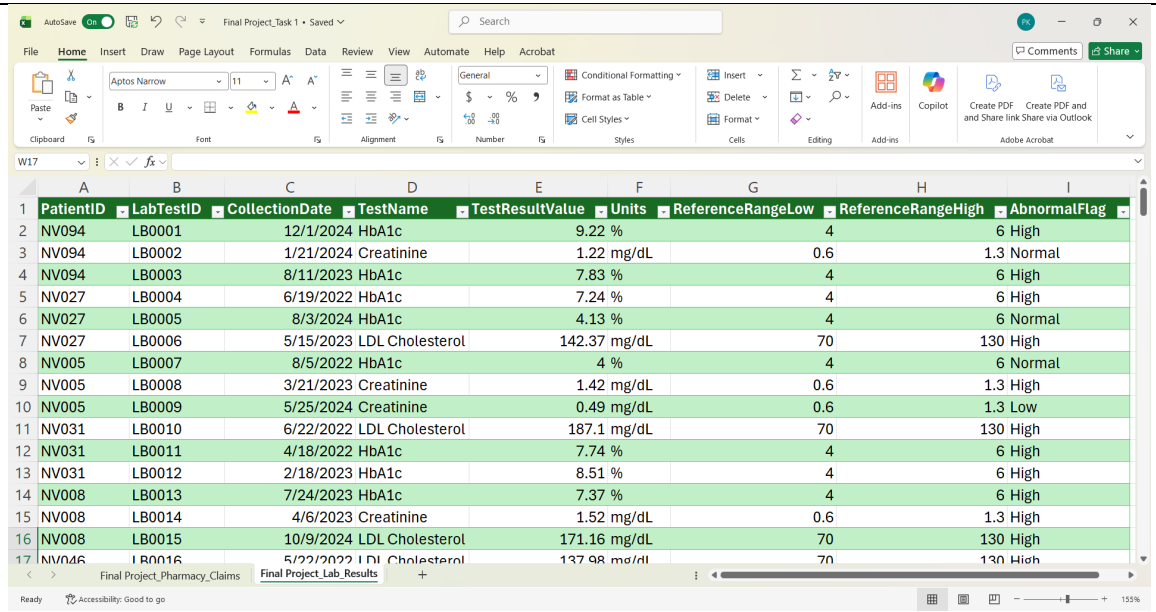
Healthcare Data Integration and Analysis

Task 1: Import the data sets from text files into spreadsheet format

Task	Activity	Screenshot of Code and Output
1a	Screenshot of *Pharmacy_Claims.xlsx*	
1b	Screenshot of *Lab_Results.xlsx*	

ClaimID	PatientID	Medication	Dosage	FillDate	Payer	ClinicID	ChargeAmount	PaidAmount
RX9000	NV094	Amlodipine	20mg	4/6/2024	Medicare	C01	159.2	97.52
RX9001	NV027	Amlodipine	5mg	4/22/2024	Medicaid	C02	381.03	325.61
RX9002	NV005	Amlodipine	10mg	1/22/2024	Medicaid	C05	296.82	215.41
RX9003	NV031	Amlodipine	500mg	8/20/2024	Private	C05	245.48	197.23
RX9004	NV008	Lisinopril	?	4/12/2024	Medicare	C02	75.07	72.29
RX9005	NV027	Amlodipine	20mg	3/4/2024	Medicaid	C03	348.16	339.67
RX9006	NV027	Lisinopril	500mg	11/10/2024	Medicaid	C02	370.47	225.59
RX9007	NV046	Amlodipine	500mg	10/13/2024	Medicaid	C02	159.6	147.28
RX9008	NV089	Lisinopril	500mg	11/12/2024	Medicare	C05	108.76	69.57
RX9009	NV072	Amlodipine	10mg	9/24/2024	Private	C04	180.2	137.94
RX9010	NV017	Lisinopril	?	6/25/2024	Medicare	C01	235.21	173.27
RX9011	NV012	Metformin	20mg	6/11/2024	Medicaid	C03	338.45	282.68



PatientID	LabTestID	CollectionDate	TestName	TestResultValue	Units	ReferenceRangeLow	ReferenceRangeHigh	AbnormalFlag
NV094	LB0001	12/1/2024	HbA1c	9.22	%	4	6	High
NV094	LB0002	1/21/2024	Creatinine	1.22	mg/dL	0.6	1.3	Normal
NV094	LB0003	8/11/2023	HbA1c	7.83	%	4	6	High
NV027	LB0004	6/19/2022	HbA1c	7.24	%	4	6	High
NV027	LB0005	8/3/2024	HbA1c	4.13	%	4	6	Normal
NV027	LB0006	5/15/2023	LDL Cholesterol	142.37	mg/dL	70	130	High
NV005	LB0007	8/5/2022	HbA1c	4	%	4	6	Normal
NV005	LB0008	3/21/2023	Creatinine	1.42	mg/dL	0.6	1.3	High
NV005	LB0009	5/25/2024	Creatinine	0.49	mg/dL	0.6	1.3	Low
NV031	LB0010	6/22/2022	LDL Cholesterol	187.1	mg/dL	70	130	High
NV031	LB0011	4/18/2022	HbA1c	7.74	%	4	6	High
NV031	LB0012	2/18/2023	HbA1c	8.51	%	4	6	High
NV008	LB0013	7/24/2023	HbA1c	7.37	%	4	6	High
NV008	LB0014	4/6/2023	Creatinine	1.52	mg/dL	0.6	1.3	High
NV008	LB0015	10/9/2024	LDL Cholesterol	171.16	mg/dL	70	130	High
NV046	LB0016	5/22/2022	LDL Cholesterol	137.98	mg/dL	70	130	High

Task 2: Analyze the data sets using Excel

2.1a	Considering the Pharmacy_Claims data set, how many paid amounts exceed $250?	**21**
2.1b	Considering the Pharmacy_Claims data set, which payer has paid the highest amount?	**Medicaid**

2.1c	Considering the *Pharmacy_Claims* data set, what is the total amount paid by the payer identified in *part 2.1b*.	**$18,396.07**
2.2a	Based on the *Lab_Results* data set, what is the top result within the LDL Cholesterol test.	**187.1 mg/dL**
2.2b	Based on the *Lab_Results* data set, what is the average value for the LDL Cholesterol test?	**134.6391 mg/dL**
2,2c	Based on the *Lab_Results* data set, how many patients have high LDL Cholesterol (> 160 mg/dL)??	**12**

Task 3: Identify and fix the issues in a data set using Python (Note: For tables including more than ten rows, only the first few rows of output are shown.)

Activity number	Activity name	Screenshot of Output

3.1a	Write the Python code to identify the number of missing values.	`print(df.isnull().sum())`
		
		`PatientID 0`
		`DateOfBirth 3`
		`Gender 0`
		`ZipCode 4`
		`PrimaryCondition 0`
		`AdmissionDate 0`
		`dtype: int64`

3.1b	Write the Python code to identify the number of duplicate values.	

```python
dup_rows = df.duplicated(keep=False)
print(f"\nNumber of duplicate rows: {dup_rows.sum()}")
if dup_rows.any():
    display(df[dup_rows])
```

```
Number of duplicate rows: 4
```

	PatientID	DateOfBirth	Gender	ZipCode	PrimaryCondition	AdmissionDate
20	NV028	18-05-1978	Male	NaN	Type 2 Diabetes (E11)	27-10-2022
21	NV028	18-05-1978	Male	NaN	Type 2 Diabetes (E11)	27-10-2022
43	NV053	1941-04-03	Male	39856.0	Hyperlipidemia (E78.5)	17-06-2024
44	NV053	1941-04-03	Male	39856.0	Hyperlipidemia (E78.5)	17-06-2024

3.1c	Write the Python code to identify inconsistent categorical entries in the Gender column.	

```python
print("\nUnique values in Gender column:")
print(df['Gender'].unique())
```

```
Unique values in Gender column:
['Female' 'Male' 'M' 'male' 'F']
```

3.1d	Write the Python code to identify inconsistent date formats in the AdmissionDate column.	`print("\nAdmissionDate(showing mixed formats):")`
		`display(df['AdmissionDate'].sample(10, random_state=42))`
		
		AdmissionDate(showing mixed formats):
		AdmissionDate
		4 17-06-2024
		63 05-03-2024
		18 17-09-2022
		0 07-06-2024
		28 11-06-2023
		73 17-12-2023
		10 23-05-2023
		34 06-07-2023
		12 19-03-2022
		55 24-04-2023

3.2a	Write the Python code to handle missing values appropriately.	

		`df['DateOfBirth'] = pd.to_datetime(df['DateOfBirth'], errors='coerce')`
		`df['ZipCode'] = pd.to_numeric(df['ZipCode'], errors='coerce')`
		
		`# Impute missing DateOfBirth with its median`
		`df['DateOfBirth'] = df['DateOfBirth'].fillna(df['DateOfBirth'].median())`
		`df['ZipCode'] = df['ZipCode'].fillna(df['ZipCode'].median())`
		
		`print("Missing values after type conversion and imputation:")`
		`print(df.isnull().sum())`
		
		`Missing values after type conversion and imputation:`
		`PatientID 0`
		`DateOfBirth 0`
		`Gender 0`
		`ZipCode 0`
		`PrimaryCondition 0`
		`AdmissionDate 0`
		`dtype: int64`

3.2b	Write the Python code to remove exact duplicate rows.	`rows_before = len(df)`
		`df.drop_duplicates(keep='first', inplace=True)`
		`rows_after = len(df)`
		`duplicated_rows_removed = rows_before - rows_after`
		
		`print(f"Number of rows before removing duplicates: {rows_before}")`
		`print(f"Number of rows after removing duplicates: {rows_after}")`
		`print(f"Number of duplicated rows removed: {duplicated_rows_removed}")`
		
		`Number of rows before removing duplicates: 74`
		`Number of rows after removing duplicates: 72`
		`Number of duplicated rows removed: 2`

<table>
<tr>
<td>3.2c</td>
<td>Write the Python code to standardize gender values.</td>
<td>

```python
df['Gender'] = df['Gender'].replace({'M': 'Male', 'm': 'Male', 'F': 'Female', 'f': 'Female', 'male': 'Male'})

print("Unique values in Gender column after standardization:")
print(df['Gender'].unique())
```

```
Unique values in Gender column after standardization:
['Female' 'Male']
```

</td>
</tr>
<tr>
<td>3.2d</td>
<td>Write the Python code to standardize date values.</td>
<td>

```python
# Parse dates with mixed formats (YYYY-MM-DD and DD/MM/YYYY)
df['AdmissionDate_parsed'] = pd.to_datetime(
    df['AdmissionDate'],
    errors='coerce',  # Convert unparseable dates to NaT (Not a Time)
    dayfirst=True     # Assume day comes first in ambiguous formats
)

print("Date parsing results:")
print(f"Successfully parsed: {df['AdmissionDate_parsed'].notna().sum()} dates")
print(f"Failed to parse: {df['AdmissionDate_parsed'].isna().sum()} dates")

print("\nSample of original vs parsed dates:")
display(df[['AdmissionDate', 'AdmissionDate_parsed']].head(5))
```

```
Date parsing results:
Successfully parsed: 72 dates
Failed to parse: 2 dates

Sample of original vs parsed dates:
```

	AdmissionDate	AdmissionDate_parsed
0	07-06-2024	2024-06-07
1	27-10-2022	2022-10-27
2	30-10-2024	2024-10-30
3	05-07-2022	2022-07-05
4	17-06-2024	2024-06-17

</td>
</tr>
</table>

Task 4: Analyze the data sets using SQL

Activity number	Activity	Screenshot of Code and Output
4a	Write an SQL query to list all patients born before 1980.	`run_query("SELECT * FROM patients WHERE DateOfBirth <'1980-01-01';")`

	PatientID	DateOfBirth	Gender	ZipCode	PrimaryCondition	AdmissionDate
0	NV094	1962-11-05 00:00:00	Female	95982	Osteoarthritis (M15.9)	2023-11-06 00:00:00
1	NV031	1976-07-08 00:00:00	Female	26647	Asthma (J45.909)	2024-08-07 00:00:00
2	NV008	1949-07-08 00:00:00	Male	56844	Hyperlipidemia (E78.5)	2023-11-28 00:00:00
3	NV046	1949-07-07 00:00:00	Male	26372	Asthma (J45.909)	2024-05-27 00:00:00
4	NV089	1943-07-18 00:00:00	Female	87372	Osteoarthritis (M15.9)	2024-01-12 00:00:00
5	NV029	1976-08-31 00:00:00	Female	41617	Asthma (J45.909)	2022-05-21 00:00:00
6	NV017	1961-07-27 00:00:00	Male	14441	Hyperlipidemia (E78.5)	2022-02-12 00:00:00
7	NV012	1966-07-24 00:00:00	Male	67468	Hypertension (I10)	2023-12-07 00:00:00
8	NV010	1947-11-12 00:00:00	Female	10156	Osteoarthritis (M15.9)	2023-05-13 00:00:00
9	NV076	1956-02-09 00:00:00	Female	14863	Asthma (J45.909)	2024-02-27 00:00:00
10	NV036	1976-12-16 00:00:00	Male	76999	Type 2 Diabetes (E11)	2022-11-28 00:00:00
11	NV063	1977-10-18 00:00:00	Female	44034	Asthma (J45.909)	2022-09-02 00:00:00

Activity number	Activity	Screenshot of Code and Output
4b	Write an SQL query to list all *female* patients with *Type 2 Diabetes*.	`run_query("SELECT * FROM patients WHERE Gender = 'Female' AND PrimaryCondition = 'Type 2 Diabetes (E11)';")`

	PatientID	DateOfBirth	Gender	ZipCode	PrimaryCondition	AdmissionDate
0	NV072	1992-11-01 00:00:00	Female	12050	Type 2 Diabetes (E11)	2023-09-20 00:00:00
1	NV001	1983-03-11 00:00:00	Female	30933	Type 2 Diabetes (E11)	2024-06-07 00:00:00
2	NV061	1986-07-06 00:00:00	Female	48060	Type 2 Diabetes (E11)	2024-04-20 00:00:00
3	NV081	1968-12-19 00:00:00	Female	19045	Type 2 Diabetes (E11)	2024-03-25 00:00:00
4	NV086	1971-08-06 00:00:00	Female	54160	Type 2 Diabetes (E11)	2022-04-18 00:00:00
5	NV062	1957-01-02 00:00:00	Female	37775	Type 2 Diabetes (E11)	2024-11-14 00:00:00

4c	Write an SQL query to display all lab tests performed. List each patient (PatientID, Gender, and DateOfBirth from *patients*) with their lab test name and result value (TestName and TestResultValue from *lab_results*).	

```
run_query("""SELECT p.PatientID, p.Gender, p.DateOfBirth, l.TestName, l.TestResultValue
FROM patients p
INNER JOIN lab_results l ON p.PatientID = l.PatientID;""")
```

	PatientID	Gender	DateOfBirth	TestName	TestResultValue
0	NV094	Female	1962-11-05 00:00:00	Creatinine	1.22
1	NV094	Female	1962-11-05 00:00:00	HbA1c	7.83
2	NV094	Female	1962-11-05 00:00:00	HbA1c	9.22
3	NV027	Male	1997-12-28 00:00:00	HbA1c	4.13
4	NV027	Male	1997-12-28 00:00:00	HbA1c	7.24
...	...	...	...	...	...
196	NV065	Male	1984-09-09 00:00:00	HbA1c	5.56
197	NV065	Male	1984-09-09 00:00:00	LDL Cholesterol	102.72
198	NV075	Female	1944-08-03 00:00:00	Creatinine	0.92
199	NV075	Female	1944-08-03 00:00:00	HbA1c	4.02
200	NV075	Female	1944-08-03 00:00:00	HbA1c	5.32

201 rows × 5 columns

4d	Write the SQL query to display all distinct patients with their payer(s). List each patient (PatientID, Gender, DateofBirth from *patients*) with their payer(s) from the *pharmacy_claims* data set.	

```
run_query("""SELECT DISTINCT p.PatientID, p.Gender, p.DateOfBirth, pc.Payer
FROM patients p
INNER JOIN pharmacy_claims pc ON p.PatientID = pc.PatientID;""")
```

	PatientID	Gender	DateOfBirth	Payer
0	NV094	Female	1962-11-05 00:00:00	Medicare
1	NV027	Male	1997-12-28 00:00:00	Medicaid
2	NV005	Female	1984-08-25 00:00:00	Medicaid
3	NV031	Female	1976-07-08 00:00:00	Private
4	NV008	Male	1949-07-08 00:00:00	Medicare
5	NV046	Male	1949-07-07 00:00:00	Medicaid